

```
In [1]: a = 5  
a
```

```
Out[1]: 5
```

```
In [2]: a = 5.5  
type(a)
```

```
Out[2]: float
```

```
In [3]: a = 5.5  
print(type(a))
```

```
<class 'float'>
```

```
In [4]: b = [2,3]  
print(type(b))
```

```
<class 'list'>
```

```
In [5]: class laptop(): # class name  
        def config(): #method name  
            print('i5', 16) # print statment  
            print('i9', 32)
```

```
lap = laptop() #lap is object
```

```
In [6]: class laptop(): # class name  
        def config(): #method name  
            print('i5', 16) # print statment  
            print('i9', 32)
```

```
lap = laptop() #lap is object
```

```
laptop.config()
```

```
i5 16
```

```
i9 32
```

```
In [7]: class laptop(): # class name  
        def config(): #method name  
            print('i5', 16) # print statment  
            print('i9', 32)
```

```
#lap = laptop() #lap is object
```

```
laptop.config()
```

```
i5 16
```

```
i9 32
```

```
In [8]: class laptop(): # class name  
        def config(): #method name  
            print('i5', 16) # print statment  
            print('i9', 32)
```

```
lap = laptop() #lap is object

lap.config()
```

```
-----
TypeError                                Traceback (most recent call last)
Cell In[8], line 8
      4         print('i9', 32)
      6 lap = laptop() #lap is object
----> 8 lap.config()

TypeError: laptop.config() takes 0 positional arguments but 1 was given
```

```
In [9]: class laptop(): # class name
        def config(self): #method name
            print('i5', 16) # print statment
            print('i9', 32)

lap = laptop() #lap is object

lap.config()
```

```
i5 16
i9 32
```

```
In [10]: class laptop(): # class name
        def config(self): #method name
            print('i5', 16) # print statment
            print('i9', 32)

lap1 = laptop() #lap is object
lap2 = laptop()
lap3 = laptop()
lap4 = laptop()

lap1.config()
lap2.config()
lap3.config()
```

```
i5 16
i9 32
i5 16
i9 32
i5 16
i9 32
```

```
In [11]: class computer():          # computer is class (user create the class hear(user defin
        def config():              # confit is method (define some method hear so when we talk
            print('i5', '8gb') # i want to print configuration of the machine which i5

a = 8
print(type(a))

com1 = computer() #com1 is the user define object of the computer class but if you
print(type(com1))
```

```
<class 'int'>
<class '__main__.computer'>
```

In [12]: `__name__`

Out[12]: `'__main__'`

```
In [13]: class computer:
          def config():      # self is the object which user passing
              print('i5', '8gb')

          com1 = computer()

          computer.config() #hear com1 act as parameter going to the self
```

i5 8gb

```
In [14]: class computer():
          def config():      # self is the object which user passing
              print('i5', '8gb')

          com1 = computer()

          computer.config(com1) #hear com1 act as parameter going to the self
```

```
-----
TypeError                                Traceback (most recent call last)
Cell In[14], line 7
      3     print('i5', '8gb')
      5 com1 = computer()
----> 7 computer.config(com1)

TypeError: computer.config() takes 0 positional arguments but 1 was given
```

```
In [15]: class computer:
          def config(self):    # self is the object which user passing
              print('i5', '8gb')

          com1 = computer()

          computer.config(com1) #hear com1 act as parameter going to the self
```

i5 8gb

```
In [16]: class computer:
          def config(self):    # self is the object which user passing & it is the keyw
              print('i5', '8gb')

          com1 = computer()

          com1.config()
```

i5 8gb

Special Method

```
In [17]: class computer():
         def __init__(self):
             print('i7', '16gb')

         c1 = computer()

         #c1.__init__()
```

i7 16gb

```
In [18]: class computer():
         def __init__(self):
             print('i7', '16gb')

         c1 = computer()
         c3 = computer()
         c3 = computer()

         #c1.__init__()
```

i7 16gb

i7 16gb

i7 16gb

```
In [19]: class computer():
         def config(self):
             print('i7', '16gb')

         c1 = computer()
         c2 = computer()
         c3 = computer()

         c1.config()
         c2.config()
```

i7 16gb

i7 16gb

```
In [20]: class computer():

         def __init__(self):
             print('in init')

         # def config(self):
         #     print('i5', '8gb')

         com1 = computer()
         #com2 = computer()

         #com1.config()
         #com2.config()
```

in init

```
In [21]: class computer():

         def __init__(self):
```

```

        print('in init')

    def config(self):
        print('i5', '8gb')

com1 = computer()
#com2 = computer()

com1.config()
#com2.config()

```

```

in init
i5 8gb

```

```

In [22]: class computer():

    def __init__(self):
        print('in init')

    def config(self):
        print('i5', '8gb')

com1 = computer()
#com2 = computer()

com1.__init__()
com1.config()

#com2.config()

```

```

in init
in init
i5 8gb

```

```

In [23]: class computer:

    def config(self):
        print('i5', '8gb')

    def config2(self):
        print('i7', '32gb')

    def __init__(self):
        print('in init')

com1 = computer()
com2 = computer()
com3 = computer()

com1.config() # we didnt pass any argument but system bydefault consider self as an
com1.config2()
#com2.config1()

```

```

in init
in init
in init
i5 8gb
i7 32gb

```

In [24]: `class computer():`

```

    def __init__(self):
        print('in init')

    def config(self):
        print('i5', '8gb')

```

```

com1 = computer('i5', 8) # com1 has cpu i5 & ram is 8
com2 = computer('i7', 16) # com2 has cpu intel 5 & ram is 8 as we have 2 computer w

com1.config()
com2.config()

```

```

-----
TypeError                                Traceback (most recent call last)
Cell In[24], line 9
      6     def config(self):
      7         print('i5', '8gb')
----> 9 com1 = computer('i5', 8) # com1 has cpu i5 & ram is 8
     10 com2 = computer('i7', 16) # com2 has cpu intel 5 & ram is 8 as we have 2 com
puter we have different ram & cpt
     12 com1.config()

TypeError: computer.__init__() takes 1 positional argument but 3 were given

```

In [25]: `class computer:`

```

    def __init__(self,cpu,ram): # cpu & ram as argument
        print('in init')

    def config(self):
        print('i5', '8gb')
        print('i7', 16)

```

```

com1 = computer('i5', 8, 10000)
com2 = computer('i7', 16, 20000)

com1.config()
com2.config()

```

```
-----
TypeError                                Traceback (most recent call last)
Cell In[25], line 10
      7     print('i5', '8gb')
      8     print('i7', 16)
----> 10 com1 = computer('i5', 8, 10000)
      11 com2 = computer('i7', 16, 20000)
      13 com1.config()

TypeError: computer.__init__() takes 3 positional arguments but 4 were given
```

```
In [26]: class computer:

    def __init__(self,cpu,ram, price): # cpu & ram as argument
        print('in init')

    def config(self):
        print('i5', '8gb', 10000)
        print('i7', 16, 2000)

com1 = computer('i5', 8, 10000)
com2 = computer('i7', 16, 20000)

com1.config()
com2.config()
```

```
in init
in init
i5 8gb 10000
i7 16 2000
i5 8gb 10000
i7 16 2000
```

```
In [27]: class computer():

    def __init__(self,cpu,ram): # cpu & ram as argument
        self.cpu = cpu #hear self is the object & we need to refer to the object
        self.ram = ram

    def config(self):
        print('config is', self.cpu, self.ram)

com1 = computer('i5', 8)
com2 = computer('i7', 16)

com1.config()
com2.config()

#print(id(com1))
#print(id(com2))
```

```
config is i5 8
config is i7 16
```

```
In [28]: class computer():

    def __init__(self,cpu,ram, price, company): # cpu & ram as argument
```

```

        self.cpu = cpu #hear self is the object & we need to refer to the object
        self.ram = ram
        self.price = price
        self.company = company

    def config(self):
        print('config is', self.cpu, self.ram, self.price, self.company)

com1 = computer('i5', 8, 50000, 'hp')
com2 = computer('i7', 16, 80000, 'lenovo')

com1.config()
com2.config()

#print(id(com1))
#print(id(com2))

```

```

config is i5 8 50000 hp
config is i7 16 80000 lenovo

```

In [29]: `class computer():` *#computer class & every concept has a object or*

```

c1 = computer() #c1 is object which referring to clas
print(id(c1))

```

Cell In[29], line 3

```

c1 = computer() #c1 is object which referring to clas
^

```

IndentationError: expected an indented block after class definition on line 1

In [30]: `class computer:` *#computer class & every concept has a object or*
`pass`

```

c1 = computer() #c1 is object which referring to clas
c2 = computer()

print(id(c1))
print(id(c2))

```

```

2645479410096

```

```

2645479407104

```

constructor & self

In [31]: `class computer():`
 `def __init__(self):`
 `self.name = 'akash'`
 `self.age = 37`

```

c1 = computer()
c2 = computer()

print(c1.name, c1.age)
print(c2.name)

```


akash 37
akash

```
In [32]: class computer():
          def __init__(self):
              self.name = 'akash'
              self.age = 37

          c1 = computer()
          c2 = computer()

          c2.name = 'mit'
          c1.age = 38
          c1.name = 'rakesh'
          c2.age = 40

          print(c1.name, c1.age)
          print(c2.name, c2.age)
```

rakesh 38
mit 40

```
In [33]: class computer():
          def __init__(self):
              self.name = 'akash'
              self.age = 37

          c1 = computer()
          c2 = computer()

          c2.name = 'mit'
          c1.age = 38
          c1.name = 'rakesh'
          c2.age = 40

          print(c1.name, c1.age)
          print(c2.name, c2.age)

          print(id(c1))
          print(id(c2))
```

rakesh 38
mit 40
2645479435984
2645479439008

encapsulation

- variables

```
In [34]: class car:
          def __init__(self):
              self.mil = 10
              self.com = 'bmw'
```

```

c1 = car()
c2 = car()

print(c1.com, c1.mil)
print(c2.com, c2.mil)

print(id(c1))
print(id(c2))

```

```

bmw 10
bmw 10
2645479440256
2645476812432

```

```

In [35]: class car():
        def __init__(self):
            self.mil = 10
            self.com = 'bmw'

c1 = car()
c2 = car()
c3 = car()

c2.mil = 8

c1.mil = 20
c1.com = 'Maruti'

print(c1.com, c1.mil)
print(c2.com, c2.mil)

print(c3.com)

```

```

Maruti 20
bmw 8
bmw

```

```

In [36]: class car:
        def __init__(self):
            self.mil = 10
            self.com = 'bmw'

c1 = car()
c2 = car()

c2.mil = 12

print(c1.com, c1.mil)
print(c2.com, c2.mil)

```

```

bmw 10
bmw 12

```

```

In [37]: class car():

        wheels = 5

```

```

def __init__(self):
    self.mil = 10
    self.com = 'bmw'

c1 = car()
c2 = car()

c2.mil = 8

print(c1.com, c1.mil, c1.wheels)
print(c2.com, c2.mil, c2.wheels)

```

bmw 10 5

bmw 8 5

In [38]:

```

class car(): # car is class

    wheels = 5    # class variable

    def __init__(self): # special method
        self.mil = 10 # instance variance (mil)
        self.com = 'bmw' # instance variable ( com)

c1 = car() #c1 is object referece to class
c2 = car() # c2 is obejct refere to car

c1.wheels= 6    #object.class varieble
c2.wheels = 3 #object.class variabele

c2.mil = 8 # obj with instance variable

print(c1.com, c1.mil, c1.wheels)
print(c2.com, c2.mil, c2.wheels)

```

bmw 10 6

bmw 8 3

INSTANCE METHOD , CLASS METHOD, Static method

In [42]:

```

class student():

    school = 'nit'    #school is class variable

    def __init__(self, m1, m2, m3) : # hear i pass 3 variable (m1,m2,m3 are instance variable)
        self.m1 = m1
        self.m2 = m2
        self.m3 = m3

    #lets create a method to calculate average of the marks
    def avg(self):    # -----> hear this average is INSTANCE METHOD
        return (self.m1+self.m2+self.m3)/3

# Lets create object hear as we created s1, s2 we created 2 object & also i want to print the average
s1 = student(10, 20, 30)
s2 = student(5, 15, 10)

```

```
print(s1.avg())
print(s2.avg())
```

20.0

10.0

```
In [43]: class student():

    school = 'nit'

    def __init__(self, m1, m2, m3) : # hear i pass 3 variable
        self.m1 = m1
        self.m2 = m2
        self.m3 = m3
    #lets create a method to calculate average of the marks
    def avg(self): # -----> hear this average is INSTANCE METHOD
        return (self.m1+self.m2+self.m3)/3

    def get_m1(self): # getter method (getter - get the value) GETTER only fetch t
        return self.m1
    def set_m1(self,value): #setter method (setter - set the value) - SETTER only s
        self.m1 = value

# Lets create object hear as we created s1, s2 we created 2 object & also i want t
s1 = student(10, 20, 30)
s2 = student(5, 15, 10)

print(s1.avg())
print(s2.avg())
```

20.0

10.0

```
In [44]: class student():

    school = 'nit'

    def __init__(self, m1, m2, m3) : # hear i pass 3 variable
        self.m1 = m1
        self.m2 = m2
        self.m3 = m3
    #lets create a method to calculate average of the marks
    def avg(self): # -----> hear this average is INSTANCE METHOD
        return (self.m1+self.m2+self.m3)/3

    def info(cls): # user need to define cls - for class variable
        return cls.school

# Lets create object hear as we created s1, s2 we created 2 object & also i want t
s1 = student(10, 20, 30)
s2 = student(5, 15, 10)

print(s1.school)
print(s1.m1)
print(s2.m2)
```

```
print(s1.avg())
print(s2.avg())

print(s1.info())
print(s2.info())
```

```
nit
10
15
20.0
10.0
nit
nit
```

```
In [45]: class student():

    school = 'nit'

    def __init__(self, m1, m2, m3) : # hear i pass 3 variable
        self.m1 = m1
        self.m2 = m2
        self.m3 = m3
    #lets create a method to calculate average of the marks
    def avg(self): # -----> hear this average is INSTANCE METHOD
        return (self.m1+self.m2+self.m3)/3

    @classmethod
    def info(cls): # user need to define cls - for class variable
        return cls.school

# Lets create object hear as we created s1, s2 we created 2 object & also i want t
s1 = student(10, 20, 30)
s2 = student(5, 15, 10)

s1.info()
```

Out[45]: 'nit'

```
In [46]: class student:

    school = 'nit'

    def __init__(self, m1, m2, m3) : # hear i pass 3 variable
        self.m1 = m1
        self.m2 = m2
        self.m3 = m3
    #lets create a method to calculate average of the marks
    def avg(self): # -----> hear this average is INSTANCE METHOD
        return (self.m1+self.m2+self.m3)/3

    @classmethod
    def info(cls):
        return cls.school

# Lets create object hear as we created s1, s2 we created 2 object & also i want t
s1 = student(34, 47, 50)
```

```
s2 = student(89, 32, 12)

print(s1.avg(), student.info(), s1.info())
print(s2.avg(), student.info(), s2.info())
```

43.666666666666664 nit nit

44.333333333333336 nit nit

In [47]: **class** student():

```
    school = 'nit'

    def __init__(self, m1, m2, m3) : # hear i pass 3 variable
        self.m1 = m1
        self.m2 = m2
        self.m3 = m3
#lets create a method to calculate average of the marks
    def avg(self): # -----> hear this average is INSTANCE METHOD
        return (self.m1+self.m2+self.m3)/3

    @classmethod
    def getSchool(cls):
        return cls.school

    @staticmethod # this student class nothing to do any class & for that also u ne
    def info():
        print('this is student class')

# Lets create object hear as we created s1, s2 we created 2 object & also i want t
s1 = student(10, 20, 30)
s2 = student(5, 15, 10)

print(s1.avg())
print(s2.avg())

print(student.getSchool())

student.info()
```

20.0

10.0

nit

this is student class

In [48]: **class** student():

```
    school = 'nit'

    def __init__(self, m1, m2, m3,m4) : # hear i pass 3 variable
        self.m1 = m1
        self.m2 = m2
        self.m3 = m3
        self.m4 = m4
#lets create a method to calculate average of the marks
    def avg(self): # -----> hear this average is INSTANCE METHOD
        return (self.m1+self.m2+self.m3+self.m4)/4
```

```
# @classmethod
# def getSchool(cls):
#     return cls.school

@staticmethod # this student class nothing to do any class & for that also u ne
def info():
    print('this is student class')

# Lets create object hear as we created s1, s2 we created 2 object & also i want t
s1 = student(10, 20, 30,40)
s2 = student(5, 15, 10,40)

print(s1.avg())
print(s2.avg())

#print(student.getSchool())

student.info()
```

25.0

17.5

this is student class

In []:

```
In [49]: class student():
        def __init__(self, name, rollno):
            self.name = name
            self.rollno = rollno

s1 = student('a',1)
s2 = student('b',2)

print(s1.name, s1.rollno)
print(s2.name, s2.rollno)
```

a 1

b 2

```
In [50]: class student:
        def __init__(self, name, rollno):
            self.name = name
            self.rollno = rollno

        def show(self):
            print(s1.name, s2.rollno)

s1 = student('a',1, 'hp','i5', 8) # the moment when we assign the variable we need
s2 = student('b',2)

s1.show()

# in this code we can add laptop configuration, cpu, ram we need to add the object
# assign in variable over there as well but if you have many variable then we need
```

```

-----
TypeError                                Traceback (most recent call last)
Cell In[50], line 9
      6     def show(self):
      7         print(s1.name, s2.rollno)
----> 9 s1 = student('a',1, 'hp','i5', 8) # the moment when we assign the variable w
e need to assign the class as well
     10 s2 = student('b',2)
     12 s1.show()

TypeError: student.__init__() takes 3 positional arguments but 6 were given

```

```

In [51]: class student:
        def __init__(self, name, rollno, laptop, cpu, ram):
            self.name = name
            self.rollno = rollno
            self.laptop = laptop
            self.cpu = cpu
            self.ram = ram

        def show(self):
            print(s1.name, s1.rollno,s1.laptop, s1.cpu, s1.ram)
            print(s2.name, s2.rollno,s2.laptop, s2.cpu, s2.ram)

s1 = student('a',1,'lenovo','i5',8) # the moment when we assign the variable we need
s2 = student('b',2,'hp','i7',16)

s2.show()

a 1 lenovo i5 8
b 2 hp i7 16

```

```

In [52]: class student:
        def __init__(self, name, rollno):
            self.name = name
            self.rollno = rollno

            self.lap = self.LAPTOP()

        def show(self):
            print(self.name, self.rollno)

        class LAPTOP:
            def __init__(self):
                self.brand = 'avita'
                self.cpu = 'i5'
                self.ram = 8

s1 = student('a',1) # the moment when we assign the variable we need to assign the
s2 = student('b',2)

s1.show()
s2.show()

lap1 = s1.lap.cpu
lap2 = s2.lap.ram

```



```
lap3 = s2.lap.brand
```

```
print(lap1)
print(lap2)
print(lap3)
```

```
a 1
b 2
i5
8
avita
```

Inheritance

```
In [53]: class A() :

    def feature1(self):          # Lets take example class A has 2 feature1 & featur
        print('feature1 is hi')

    def feature2(self):
        print('feature2 is hi')

# the way you access the feature is object

a1 = A() #a1 is the object & A is constructor & using a1 we can call the feature1

a1.feature1()
a1.feature2()
```

```
feature1 is hi
feature2 is hi
```

```
In [55]: class A():
    def feature1(self):
        print('feature1 is hi')
    def feature2(self):
        print('feature2 is hi')

class B():
    def feature3(self):
        print('feature3 is hallo')
    def feature4(self):
        print('feature4 is hallo')

a1 = A() #a1 is the object & A is constructor & using a1 we can call the feature1
a1.
a1.

b1 = B() #a1 is the object & A is constructor & using a1 we can call the feature1

b1.
b1.
```

Cell In[55], line 14

a1.

^

SyntaxError: invalid syntax

```
In [58]: class A():
    def feature1(self):
        print('feature1 is hi')
    def feature2(self):
        print('feature2 is hi')

    class B():
        def feature3(self):          # Lets take example class A has 2 feature1 & featur
            print('feature3 is hallo')
        def feature4(self):
            print('feature4 is hallo')

    class C(A, B):
        def feature5(self):
            print('feature5 is how are you')

    # the way you access the feature is object

    a1 = A()  #a1 is the object & A is constructor & using a1 we can call the feature1
    b1 = B()  # INHERITENCE CUZ THE CLASS B IS INHERIT THE CLASS
    c1 = C()  # class C is inherit the class A
```

```
In [59]: class A(): #Lets create a class hear it should be employee class or student class &
    def __init__(self):
        print('in the init')
    def feature1(self):          # Lets take example class A has 2 feature1 & featur
        print('feature1 is hi')
    def feature2(self):
        print('feature2 is hi')

    class B():
        def feature3(self):          # Lets take example class A has 2 feature1 & featur
            print('feature3 is hallo')
        def feature4(self):
            print('feature4 is hallo')

    a1 = A()
```

in the init

```
In [60]: class A(): #Lets create a class hear it should be employee class or student class &
    def __init__(self):
        print('in the init')
    def feature1(self):          # Lets take example class A has 2 feature1 & featur
        print('feature1 is hi')
    def feature2(self):
        print('feature2 is hi')

    class B(A):
        def feature3(self):          # Lets take example class A has 2 feature1 & featur
            print('feature3 is hallo')
```

```

def feature4(self):
    print('feature4 is hallo')

a1 = A()
b1 = B()

```

in the init
in the init

In [61]:

```

class A(): #lets create a class hear it should be employee class or student class &
    def __init__(self):
        print('in the init')
    def feature1(self):          # Lets take example class A has 2 feature1 & featur
        print('feature1 is hi')
    def feature2(self):
        print('feature2 is hi')

class B():
    def __init__(self):
        print('in the B init')
    def feature3(self):          # Lets take example class A has 2 feature1 & featur
        print('feature3 is hallo')
    def feature4(self):
        print('feature4 is hallo')

a1 = A()
b1 = B()

```

in the init
in the B init

In [62]:

```

class A(): #lets create a class hear it should be employee class or student class &
    def __init__(self):
        print('in the init')
    def feature1(self):          # Lets take example class A has 2 feature1 & featur
        print('feature1 is hi')
    def feature2(self):
        print('feature2 is hi')

class B(A):
    def feature3(self):          # Lets take example class A has 2 feature1 & featur
        print('feature3 is hallo')
    def feature4(self):
        print('feature4 is hallo')

b1 = B()

```

in the init

In [63]:

```

class A : #lets create a class hear it should be employee class or student class &
    def __init__(self):
        print('in the A init')
    def feature1(self):          # Lets take example class A has 2 feature1 & featur
        print('feature1 is hi')
    def feature2(self):
        print('feature2 is hi')

```

```
class B():
    def __init__(self):
        print('in the B init')
    def feature3(self):          # Lets take example class A has 2 feature1 & featur
        print('feature3 is hallo')
    def feature4(self):
        print('feature4 is hallo')

b1 = B()
```

in the B init

In [64]:

```
class A() : #Lets create a class hear it should be employee class or student class
    def __init__(self):
        print('in the A init')
    def feature1(self):          # Lets take example class A has 2 feature1 & featur
        print('feature1 is hi')
    def feature2(self):
        print('feature2 is hi')

class B(A):
    def __init__(self):
        print('in the B init')
    def feature3(self):          # Lets take example class A has 2 feature1 & featur
        print('feature3 is hallo')
    def feature4(self):
        print('feature4 is hallo')

b1 = B()
```

in the B init

In [65]:

```
class A : #Lets create a class hear it should be employee class or student class &
    def __init__(self):
        print('in the A init')
    def feature1(self):          # Lets take example class A has 2 feature1 & featur
        print('feature1 is hi')
    def feature2(self):
        print('feature2 is hi')

class B(A):
    def __init__(self):
        super().__init__()
        print('in the B init')
    def feature3(self):          # Lets take example class A has 2 feature1 & featur
        print('feature3 is hallo')
    def feature4(self):
        print('feature4 is hallo')

b1 = B()
```

in the A init

in the B init

In [66]:

```
class A() : #Lets create a class hear it should be employee class or student class
    def __init__(self):
        print('in the A init')
```

```

def feature1(self):          # Lets take example class A has 2 feature1 & featur
    print('feature 1-A is hi')
def feature2(self):
    print('feature2 is hi')

class B():
    def __init__(self):
        print('in the B init')
    def feature3(self):       # Lets take example class A has 2 feature1 & featur
        print('feature 3-B is hallo')
    def feature4(self):
        print('feature4 is hallo')

class C(A,B):
    def __init__(self):
        super().__init__()
        print('in the C init')

c1 = C()
c1.

```

Cell In[66], line 23

c1.

^

SyntaxError: invalid syntax

In [67]:

```

class A : #Lets create a class hear it should be employee class or student class &
    def __init__(self):
        print('in the A init')
    def feature1(self):          # Lets take example class A has 2 feature1 & featur
        print('feature 1-A is hi')
    def feature2(self):
        print('feature2 is hi')

class B():
    def __init__(self):
        print('in the B init')
    def feature3(self):       # Lets take example class A has 2 feature1 & featur
        print('feature 3-B is hallo')
    def feature4(self):
        print('feature4 is hallo')

class C(A,B):
    def __init__(self):
        super().__init__()
        print('in the C init')
    def feat(self):
        super().feature2()

c1 = C()

```

in the A init

in the C init

duck typing

```
In [68]: class duck():
    def swim(self):
        print('i am duck & i can swim')
    def speaks(self):
        print('quack quack')

class dog():
    def swim(self):
        print('i am dog & i can swim')
    def speaks(self):
        print('woof woof')

def display(duck): # we define outside of the class
    duck.swim()
    duck.speaks()
    #print('informatation')

d = duck()
display(d)
```

i am duck & i can swim
quack quack

```
In [69]: class duck:
    def swim(self):
        print('i am duck & i can swim')
    def speaks(self):
        print('quack quack')

class dog:
    def swim(self):
        print('i am dog & i can swim')
    def speaks(self):
        print('woof woof')

def display(duck): # we define outside of the class
    duck.swim()
    duck.speaks()
    #print('informatation')

d = duck()
dg = dog()

display(dg)
```

i am dog & i can swim
woof woof

```
In [70]: class duck:
    def swim(self):
        print('i am duck & i can swim')
    def speaks(self):
        print('quack quack')
```

```

class dog:
    def swim(self):
        print('i am dog & i can swim')
    def speaks(self):
        print('woof woof')

class person:
    def speaks(self):
        print('attend more interview to get placed')

def display(obj): # we define outside of the class
    obj.swim()
    obj.speaks()
    obj.person()

d = duck()
dg = dog()
p = person()

display(p)

```

```

-----
AttributeError                                Traceback (most recent call last)
Cell In[70], line 26
     23 dg = dog()
     24 p = person()
--> 26 display(p)

Cell In[70], line 18, in display(obj)
     17 def display(obj): # we define outside of the class
--> 18     obj.swim()
     19     obj.speaks()
     20     obj.person()

AttributeError: 'person' object has no attribute 'swim'

```

```

In [71]: class duck:
        def swim(self):
            print('i am duck & i can swim')
        def speaks(self):
            print('quack quack')

class dog:
    def swim(self):
        print('i am dog & i can swim')
    def speaks(self):
        print('woof woof')

class person:
    def speaks(self):
        print('attend more interview to get placed')
    def swim(self):
        print('need to learn for swimming')

def display(obj): # we define outside of the class
    obj.swim()

```

```
obj.speaks()  
#obj.person()  
  
d = duck()  
dg = dog()  
p = person()  
  
display(p)
```

need to learn for swimming
attend more interview to get placed

In []:

In []:

In []:

In []:

In []:

In []: